Arquitetura de Software B1

Equipe

Gustavo Gutierres Champam - RA 223645
Gustavo Oliveira Camargo - RA 236024

Decisão geral

Evoluir o Core para monólito modular, isolar ranking com Ports & Adapters e separar IA em um serviço Python. Eventos usam outbox e dispositivos usam MQTT com gateway. As seis decisões são rastreáveis a RFs, RNFs, ASRs e restrições.

Escopo da evidência

Código Java compilado e testado, integração real Spring/FastAPI, mini-labs de outbox e MQTT e coleta Prometheus/Grafana. O material usa o Core de referência fornecido. Não constitui teste de capacidade para 100 mil usuários nem comprovação de execução no Jenkins.

Organização

O registro principal está em docs/arquitetura/README.md. ADRs em docs/adr; matriz em docs/arquitetura/matriz_asr.csv; código em core, ia e microsolucoes; logs e capturas em evidencias.

Desafio 1 Drivers e evolução do Core

RF e RNF

RF01 Ranking e RF02 Conquistas. O cenário de crescimento para até 100 mil usuários representa uma previsão de capacidade, não uma medição de simultaneidade. RNFs: manutenibilidade (S), desempenho e escalabilidade (P), testabilidade (S) e complexidade operacional (S). Testabilidade é importante, mas os três ASRs selecionados abaixo são os que dirigem a escolha macro.

ASRs prioritários

D1-A1 - Manutenibilidade (S): Localizar mudanças no módulo de ranking, restringindo acesso à persistência. Verificação: Domínio de pontuação sem imports de Spring/JPA; revisão de dependências.
D1-A2 - Escalabilidade (P): Preservar fronteira para extrair ranking apenas se a carga justificar outro deploy. Verificação: Meta proposta: p95 de leitura < 300 ms sob carga definida na AF; não validada para 100 mil usuários.
D1-A3 - Complexidade operacional (S): Manter um deploy e um banco para equipe de seis pessoas. Verificação: Contagem dos processos essenciais do Core: um.

RPCs

Java/Spring Boot, banco relacional, legado da AC1 e equipe de seis desenvolvedores. Como o repositório específico da AC1 não foi fornecido, a microsolução parte do monólito modular de referência; sua integração ao MVP da equipe precisa ser validada antes da entrega definitiva.

Alternativas e trade offs

A - Camadas: operação simples, mas mudanças em uma capacidade atravessam pastas horizontais. B - Monólito modular: agrupa capacidade e explicita limites mantendo uma transação local; requer disciplina de dependências. C - Extração seletiva de ranking: escala e deploy próprios, mas adiciona contrato remoto, consistência eventual e operação. O volume total de usuários, isoladamente, não prova a necessidade de C.

ADR 001 Decisão e evidência

Decisão

Escolhemos o monólito modular e mantivemos o ranking no mesmo processo. O critério de revisão é saturação comprovada do ranking, apesar de paginação, índices e otimização, ou necessidade frequente de deploy independente. A equipe de seis pessoas favorece reduzir componentes operacionais enquanto não há essa evidência.

Consequências

Um único deploy continua sendo ponto comum de falha e escala. A consulta original de ranking ainda lê todos os registros e acopla-se à assinatura do usuário; a modularização não elimina esse custo nem oferece isolamento completo. Paginação e projeção de leitura são próximos experimentos. Relações JPA entre módulos permanecem como dívida explícita.

Evidência

comparacao_dependencias.json; core/src/main/java/com/exemplo/usuario/ranking/. O projeto em camadas dispersa API, serviço e repository; o modular agrupa essas classes. A evolução adiciona porta e regra pura. Imports ausentes de Usuario/Curso em Matricula, já presentes no ZIP de referência, foram corrigidos para compilar.

Impacto C4

Container: navegador, Core e banco permanecem. Component: ranking passa a ter API, serviço, porta, regra de pontuação e adapter JPA.

Desafio 2 Organização interna do Core

RF e RNF

RF01 Ranking e RF02 regras de gamificação. RNFs: manutenibilidade, modificabilidade, testabilidade e evolução (S). Selecionamos modificabilidade e testabilidade como ASRs; não é preciso criar um terceiro apenas para preencher a matriz.

ASRs prioritários

D2-A1 - Modificabilidade (S): Inverter a dependência da aplicação para uma porta de leitura. Verificação: RankingService depende de RankingPort, não do repository.
D2-A2 - Testabilidade (S): Executar regra e caso de uso sem inicializar Spring nem banco. Verificação: Pontuacao e fake testados com JUnit; mesmo resultado com JPA/H2.

RPCs

Spring Boot permanece; regras existentes devem ser reaproveitadas; não reescrever o produto inteiro. Preservada a fórmula cursos concluídos × 100 + moedas e o endpoint de ranking.

Alternativas e trade offs

A - Serviço diretamente ligado a RankingRepository e entidade Assinatura: menos tipos, mas testes e mudanças de banco afetam a aplicação. B - Porta de leitura RankingPort e adapter JPA: a aplicação recebe dados simples, e o domínio calcula os pontos; adiciona mapeamento e uma interface. DDD ajuda a nomear capacidades e regras; Hexagonal define portas/adapters; Clean e Onion descrevem direção de dependências. Não são produtos obrigatórios nem padrões mutuamente exclusivos.

ADR 002 Decisão e evidência

Decisão

Aplicamos Ports & Adapters somente ao ranking. RankingPort pertence à aplicação; RankingJpaAdapter implementa a porta; Pontuacao é Java puro. A aplicação ainda usa @Service/@Transactional e DTO de saída: é uma adoção incremental, não uma alegação de Clean Architecture integral.

Consequências

Trocar persistência exige implementar a porta, sem mudar a regra. Existem mais tipos e mapeamentos. O repository ainda ordena pela expressão SQL equivalente à fórmula de domínio; alterar a fórmula exige atualizar a ordenação e seu teste de contrato. Um único cálculo no banco/projeção será avaliado se o volume tornar o mapeamento caro.

Evidência

RankingTest valida 320 pontos com adapter em memória; RankingJpaTest grava e recarrega assinatura com H2 e obtém os mesmos 320 pontos. Pontuacao testa negativos e overflow. A regra não importa Spring ou JPA.

Impacto C4

Component do Core mostra controller → serviço → porta; adapter JPA implementa porta e acessa repository; serviço chama Pontuacao.

Desafio 3 Serviço Python e IA

RF e RNF

RF03 recomendação e RF04 assistente educacional. RNFs: deploy independente, escalabilidade, tolerância a falhas, interoperabilidade, desempenho e manutenibilidade. Interoperabilidade HTTP/JSON é necessária, mas os ASRs prioritários são autonomia de deploy, falha e escala.

ASRs prioritários

D3-A1 - Deploy independente (S): Separar o processo Python do Core para publicar modelos sem recompilar Java. Verificação: Dockerfiles separados; endpoint FastAPI executado em processo distinto.
D3-A2 - Tolerância a falhas (R): Limitar espera da IA e responder com catálogo quando ela falhar. Verificação: HTTP 200 com origem python e, após queda, origem catalogo.
D3-A3 - Escalabilidade (P): Permitir ampliar réplicas da IA sem replicar todo o Core. Verificação: Fronteira de processo presente; teste de carga ainda necessário.

RPCs

Core em Java/Spring Boot; ML e RAG em Python; a indisponibilidade da IA não pode bloquear operações essenciais. O mini-lab usa uma resposta determinística de recomendação para testar a fronteira; não contém modelo treinado nem uma implementação completa de RAG.

Alternativas e trade offs

A - IA embutida: menos rede, porém runtime e atualização de modelos prendem-se ao Core. B - Módulo acoplado/script local: interoperação possível, mas compartilha ciclo de implantação e recursos. C - Serviço Python: contrato remoto e falhas de rede, porém deploy e capacidade independentes. Separar ML e RAG já de início duplica operação sem medição; mantê-los juntos aceita disputa interna por recursos.

ADR 003 Decisão e evidência

Decisão

Escolhemos um serviço Python independente, com endpoints e responsabilidades separáveis. ML e RAG começam no mesmo limite operacional; extrair RAG se consumo de memória, latência ou cadência de atualização divergirem. O idioma Python sozinho não justifica um serviço: a decisão depende dos três ASRs.

Consequências

Recomendação indisponível devolve catálogo básico; assistente deve informar indisponibilidade, nunca inventar resposta. Cliente Java tem timeout de conexão de 300 ms e de requisição de 800 ms, sem retries no caminho interativo. Instrumentamos um contador para cada fallback. Circuit breaker e limite de concorrência ficam para a AF, após observar falhas sustentadas.

Evidência

ia-online.log e ia-offline.log registram o mesmo endpoint Spring respondendo 200: primeiro origem python, depois origem catalogo com o processo FastAPI interrompido. O teste automatizado IaTest comprova fallback e contador em porta indisponível.

Impacto C4

Container adiciona serviço Python e HTTP/JSON Core → IA. Vetores/documentos do RAG são capacidade futura, não artefatos do mini-lab.

Desafio 4 Comunicação e eventos

RF e RNF

RF02 conclusão/recompensas e RF05 notificações. RNFs: tolerância a falhas, disponibilidade, latência, consistência, desacoplamento e recuperabilidade. Conclusão e recompensa local exigem confirmação imediata; notificação e analytics aceitam processamento eventual.

ASRs prioritários

D4-A1 - Disponibilidade (R): Retirar notificação do caminho síncrono da conclusão. Verificação: Conclusão permanece gravada com consumidor indisponível.
D4-A2 - Consistência (R): Gravar conclusão e intenção de notificar na mesma transação. Verificação: Outbox SQLite persiste junto com conclusão.
D4-A3 - Recuperabilidade (R): Reentregar pendências e deduplicar identificadores no consumidor. Verificação: Reabertura do banco recupera evento; uma única entrega lógica.

RPCs

Consumidor externo indisponível não pode impedir a conclusão; broker adiciona custo de operação. A microsolução usa SQLite para demonstrar atomicidade local e durabilidade, sem simular uma implantação completa em PostgreSQL.

Alternativas e trade offs

A - REST síncrono: resposta imediata e fluxo simples, mas a indisponibilidade externa propaga falha. B - Outbox transacional e worker com entrega HTTP: grava intenção junto ao estado, permite retry, adiciona agendamento e controle de duplicidade. C - Outbox + RabbitMQ: útil para filas, roteamento e vários workers. Kafka favorece retenção/replay e múltiplos fluxos de consumo, mas exige governança e operação desproporcionais à notificação atual.

ADR 004 Decisão e evidência

Decisão

Decidimos manter conclusão/recompensa no Core e notificação eventual via outbox. Não adicionar RabbitMQ/Kafka neste estágio. O worker deve confirmar sucesso depois do consumidor e repetir pendências. Event ID idempotente no consumidor garante um único efeito mesmo com entrega repetida.

Consequências

Sem exactly once: queda entre envio e marcação pode repetir a entrega. A AF deve persistir deduplicação no consumidor, definir backoff, limite de tentativas, fila de falhas e retenção. A notificação pode atrasar e o estado operacional de pendências precisa ser medido. O lab não altera ainda o fluxo de matrícula do Core.

Evidência

microsolucoes/eventos.py: falha síncrona é propagada; conclusão+outbox permanecem gravadas quando a entrega falha; reabrir banco e reentregar recupera a pendência; repetir o mesmo event ID produz um único registro recebido.

Impacto C4

Container alvo inclui worker de outbox e consumidor de notificação. A sequência distingue commit local de entrega posterior. Estes componentes estão no lab de eventos, ainda separados da aplicação Spring.

Desafio 5 IoT e MQTT

RF e RNF

RF06 receber eventos IoT e encaminhá-los à plataforma. RNFs: interoperabilidade, eficiência, conectividade intermitente, escalabilidade e segurança. Os três ASRs escolhidos orientam protocolo, retry/deduplicação e fronteira de confiança.

ASRs prioritários

D5-A1 - Eficiência de comunicação (P): Usar publicação MQTT com mensagens pequenas em vez de polling HTTP. Verificação: Publicador e consumidor QoS 1 executados em broker local.
D5-A2 - Conectividade intermitente (R): Exigir identificador estável, buffer e reenvio no dispositivo. Verificação: Duplicata testada; buffer persistente e sessão durável ainda são trabalho da AF.
D5-A3 - Segurança de integração (F+): Isolar tópico por instituição e validar identidade e payload no gateway. Verificação: Tópico institucional demonstrado; TLS/ACL ainda não exercitados no lab local.

RPCs

Dispositivos podem ter CPU/memória limitadas e conexão instável. Broker e protocolo adicionam operação. O mini-lab usa broker real em loopback e modo anônimo apenas em ambiente local isolado.

Alternativas e trade offs

A - REST direto: infraestrutura conhecida, mas dispositivos precisam implementar retry e conexão por requisição; Core fica exposto ao ritmo dos dispositivos. B - MQTT + gateway: publicação leve, QoS e desacoplamento; demanda broker, autenticação e gestão de sessão. C - MQTT + outro broker interno: justificável quando vários consumidores precisam replay/roteamento independente; hoje duplica a operação sem requisito demonstrado.

ADR 005 Decisão e evidência

Decisão

Escolhemos MQTT QoS 1 com gateway que transforma o contrato do dispositivo. Tópico por instituição, event_id estável e consumidor idempotente. Não adicionar outro broker. Em produção, usar TLS, credenciais por dispositivo, ACL por tópico, validação de esquema e deduplicação persistente antes de aplicar o evento no Core.

Consequências

QoS 1 admite duplicatas e não substitui buffer persistente do dispositivo. O mini-lab demonstra deduplicação em memória e transformação, mas não comprova recuperação depois de reinício nem segurança multi-instituição. Esses itens são critérios de aceitação da AF, e não capacidades alegadas como já implementadas.

Evidência

microsolucoes/mqtt_lab.py e mqtt.log: publicador envia duas cópias com QoS 1 ao broker; gateway recebe duas mensagens e gera uma PresencaRegistrada. A transformação ocorre antes da futura integração com o Core.

Impacto C4

Container alvo: dispositivo → broker MQTT → gateway → Core. MQTT/TCP no lab; MQTT/TLS e HTTPS na implantação proposta.

Desafio 6 CI CD e observabilidade

RF e RNF

Todos os RFs dependem de construção e operação confiáveis. RNFs: observabilidade, deploy independente, confiabilidade, testabilidade, recuperabilidade e desempenho. Foram selecionados observabilidade, confiabilidade de entrega e recuperabilidade operacional.

ASRs prioritários

D6-A1 - Observabilidade (S): Instrumentar falhas recuperadas, além do status HTTP. Verificação: ia_fallback_total aumenta mesmo quando a API responde 200.
D6-A2 - Confiabilidade de entrega (R): Bloquear implantação quando testes ou build falharem. Verificação: Jenkinsfile sequencial sem ignorar falhas; Maven executado localmente.
D6-A3 - Recuperabilidade operacional (S): Versionar imagens/configuração e preparar retorno à versão anterior. Verificação: JAR e configurações versionáveis; rollback em ambiente Jenkins ainda pendente.

RPCs

Jenkins é a base da esteira e Prometheus/Grafana são ferramentas do laboratório. Artefatos e configuração versionados. Esta entrega inclui a evolução pedida no B1; a atividade A2 de Jenkins não foi refeita nem sobrescrita.

Alternativas e trade offs

A - Pipeline único: integração simples e coordenação central, mas mudanças na IA podem aguardar etapas do Core. B - Pipelines por componente: deploy independente e menor escopo de falha, com custo de contratos e promoção conjunta. Com um único repositório e equipe pequena, o pipeline inicial único tem etapas separadas; extrair pipelines quando a frequência independente for comprovada.

ADR 006 Decisão e evidência

Decisão

Preparamos um Jenkinsfile com teste/build antes de Docker e smoke test após implantação. Falhas Maven bloqueiam deploy; falha de saúde falha a execução. Prometheus coleta Actuator e Grafana consulta Prometheus. O contador ia_fallback_total evidencia degradação invisível ao HTTP 200.

Consequências

Jenkinsfile fornecido pressupõe agente Linux com JDK jdk17, Python e Docker. Foi revisado, mas não executado num servidor Jenkins. Compose foi preparado, enquanto as evidências locais foram produzidas por processos nativos. Não há rollback automático implementado; a AF deve usar tags imutáveis e validar retorno à versão anterior.

Evidência

core-build.log, prometheus.png, grafana.png, grafana-fallback.png e metricas.json. Core e A3 apareceram UP; interrupção de Python gerou fallback observável. up comprova scrape e não disponibilidade funcional completa. Histogramas de HTTP permitem p95; número de usuários cadastrados não comprova throughput.

Impacto C4

Deployment distingue processos nativos usados na validação de containers planejados para o lab Jenkins. Visão final mantém Core único, IA separada e integrações de eventos/IoT explicitamente identificadas como microsoluções.

Síntese para a Avaliação Final

Decisões levadas à PoC

Para a AF, levaremos o monólito modular, a porta de ranking e a fronteira Python com timeout/fallback. Integrar a outbox ao fluxo real de conclusão e persistir a deduplicação no consumidor. Conectar o gateway ao Core, adicionar ACL/TLS e testar desconexão/reinício. Evoluir recomendação de fixture para modelo e RAG com avaliação de resposta. Validar isolamento por instituição e paginação de ranking. Executar a esteira Jenkins, testar rollback e medir p95/throughput com uma carga explicitamente definida. Não extrair ranking nem adotar Kafka antes de medir o gargalo.

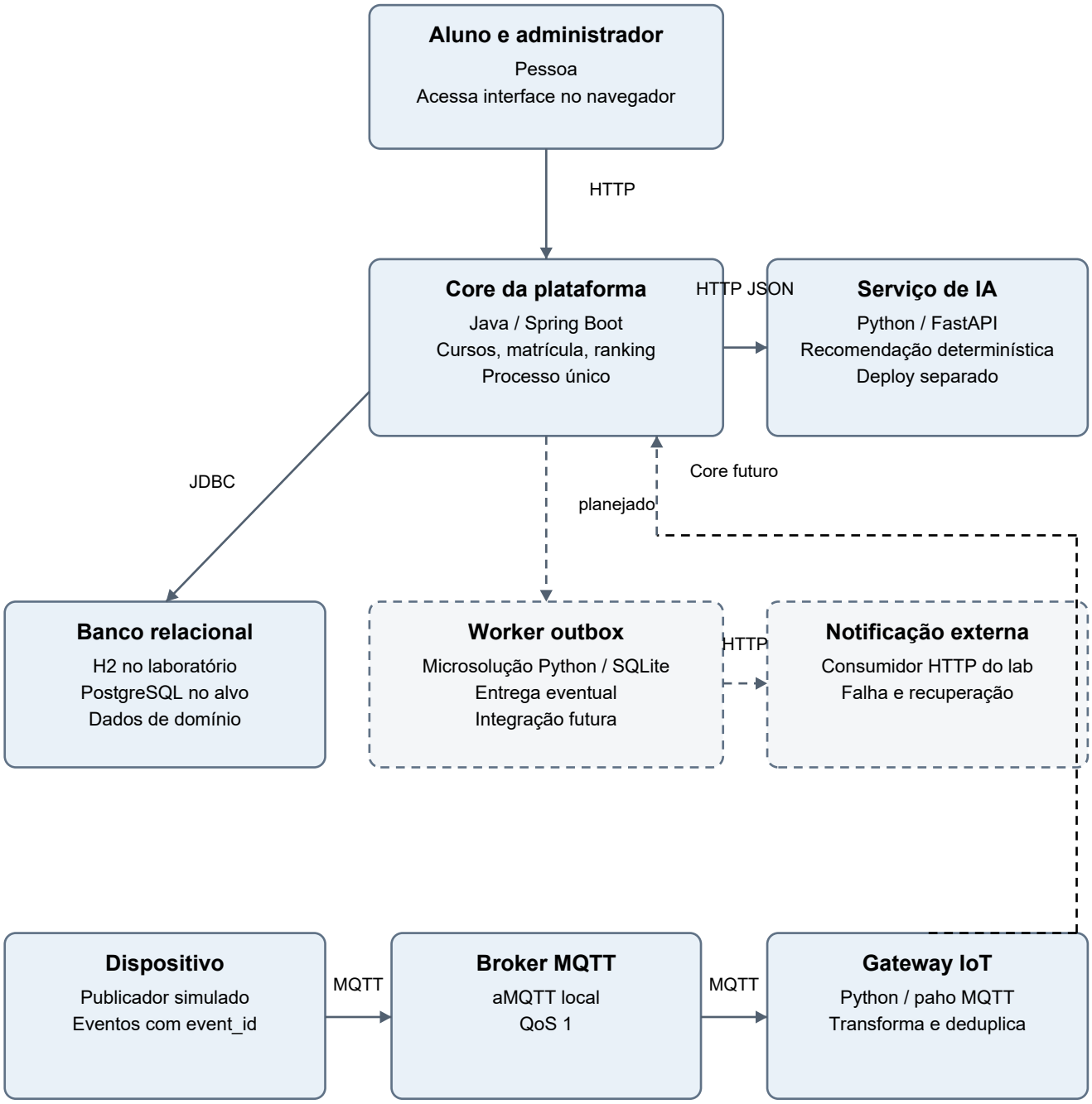
Critérios de aceitação propostos

Ranking p95 abaixo de 300 ms em carga a definir; fallback de IA em até 1,5 s no ambiente controlado; nenhuma conclusão perdida com notificação indisponível; duplicatas não geram efeito extra; recuperação de evento após reinício; scrape operacional e contador de degradação. Esses valores são propostas para validação, não resultados medidos de capacidade.

Pendências de entrega externa

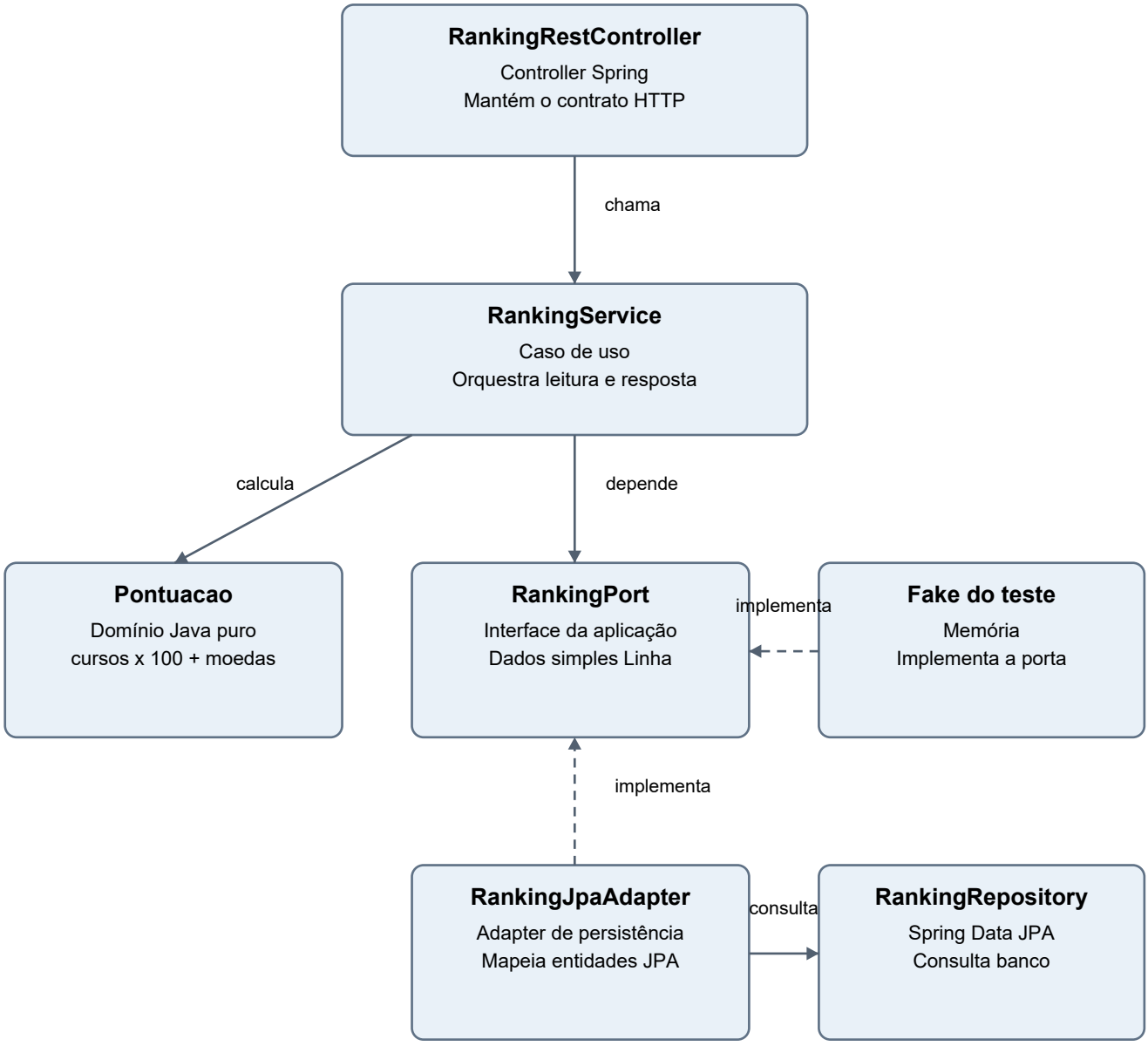
Publicar o repositório no Git da equipe e informar o link no Canvas. Confirmar aderência ao MVP da AC1. Não houve publicação externa, submissão no Canvas ou execução no Jenkins nesta preparação.

C4 Container da plataforma



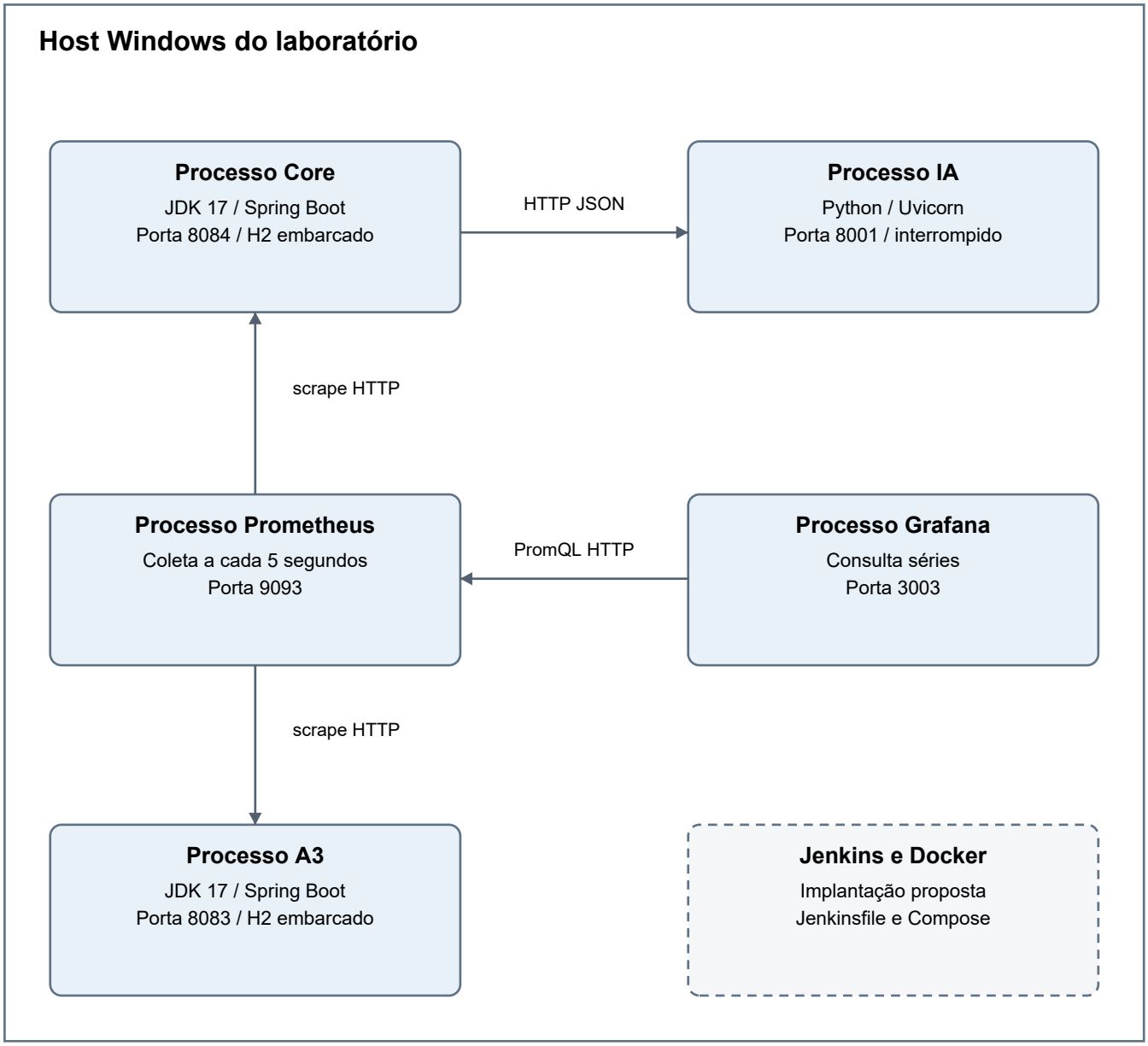
Contínuo: componente presente no laboratório. Tracejado: integração ainda planejada. Outbox e gateway são microsoluções separadas do Core; não representam fluxo de matrícula já integrado.

C4 Component do ranking



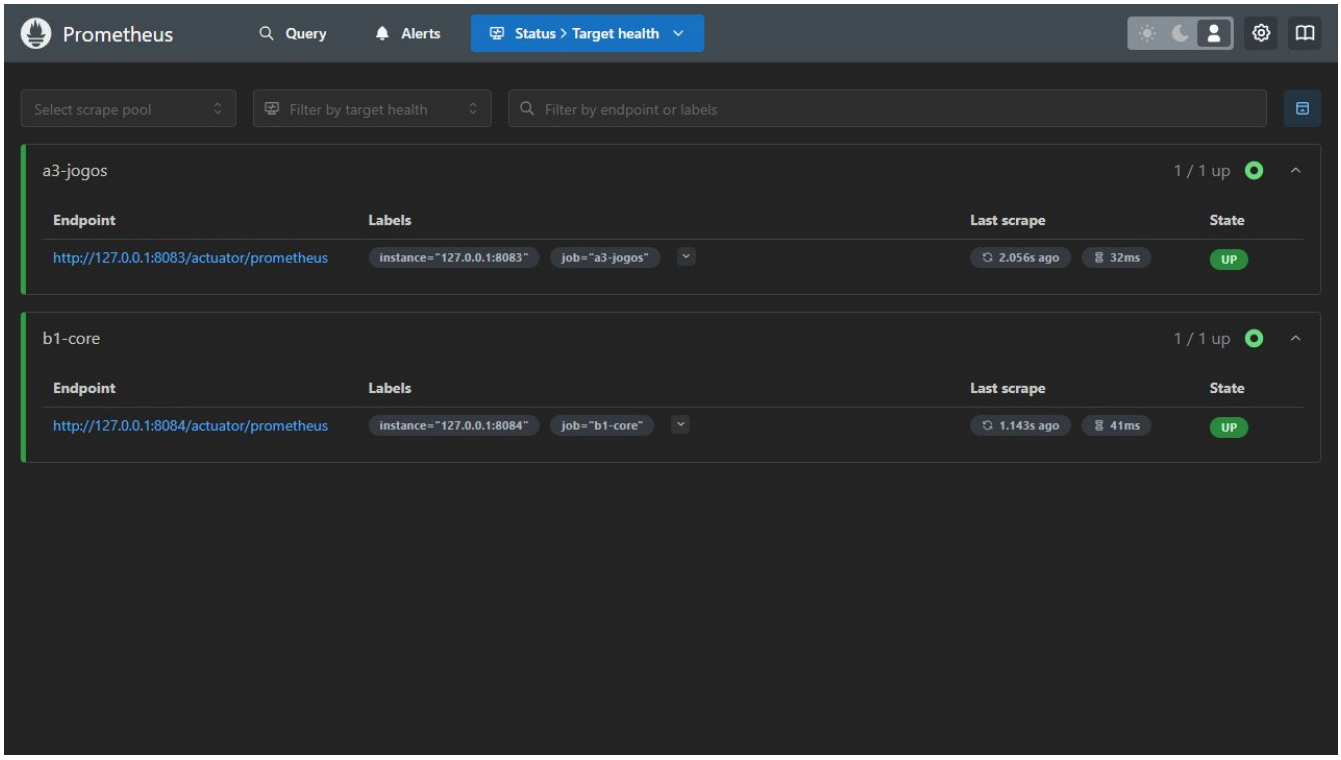
A aplicação depende de RankingPort. O adapter JPA e o fake implementam esse contrato. Pontuacao permanece sem dependência de Spring ou JPA. A adoção é incremental; outros módulos preservam a estrutura fornecida.

C4 Implantação e operação



Os processos contínuos foram executados na máquina do laboratório. Jenkins e Docker aparecem tracejados porque os arquivos foram preparados, mas essa implantação ainda não foi executada. As portas são da validação nativa; o Compose do B1 usa Prometheus 9094 e Grafana 3004.

Prometheus com coleta ativa



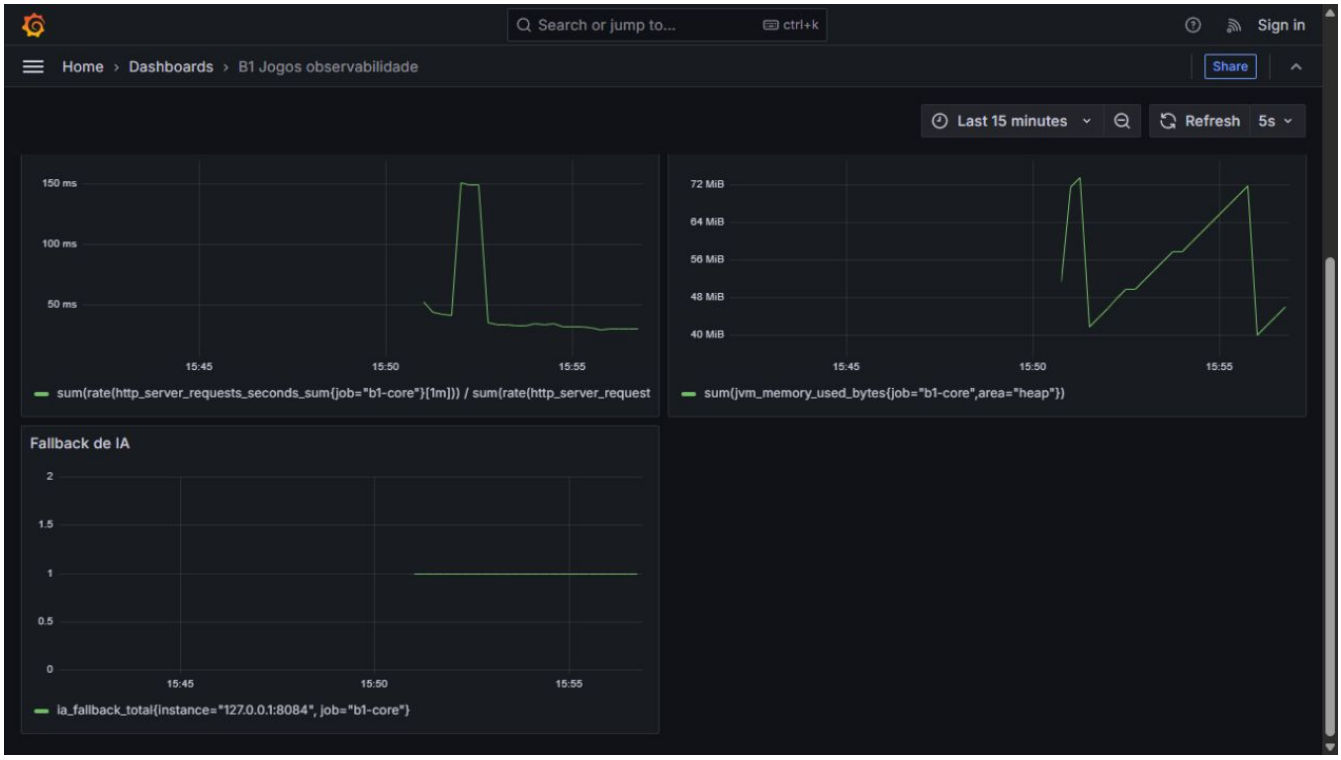
Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Dashboard Grafana



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Métrica de negócio após execução



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

O processo Python foi interrompido. A API Java retornou catálogo de fallback com HTTP 200 e incrementou ia_fallback_total. Assim, a métrica revela a degradação que o código HTTP sozinho não mostra.

Referências e rastreabilidade

Enunciados e projetos fornecidos pela disciplina.

Spring Boot 3.3 - Testing Spring Boot Applications:
https://docs.spring.io/spring-boot/3.3/reference/testing/spring-boot-applications.html

Mockito - InOrder: https://site.mockito.org/javadoc/current/org/mockito/InOrder.html

JaCoCo - check goal: https://www.jacoco.org/jacoco/trunk/doc/check-mojo.html

C4 Model: https://c4model.com/

Prometheus overview: https://prometheus.io/docs/introduction/overview/

As métricas do relatório são da execução local. Jenkins/Compose e capacidade de produção não foram validados. Os arquivos Markdown mantêm o conteúdo editável e os comandos necessários para repetir os experimentos.